

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

25 - Multiplication

Introduction

- Arithmetic operation
- More vexing operation

Multiplication Components

- Multiply two operands
 - Multiplicand and
 - Multiplier,
- Result, called Product
- To express relationship between the components:
$$\text{Product} = \text{multiplicand} \times \text{multiplier}$$

Decimal Multiplication

- Review multiplication of decimal numbers in longhand to recall the steps and the names of the operands
 - Multiplying 1000_{ten} by 1001_{ten} :

Multiplicand		1000_{ten}
Multiplier	x	1001_{ten}
		1000
		0000
		0000
		1000
Product		1001000_{ten}

Observations

- Number of digits in product is larger than number in either multiplicand or multiplier
- If ignore sign bits, **length** of multiplication of **n -bit** multiplicand and **m -bit** multiplier is a product that is $n + m$ bits
 - $n + m$ bits are required to represent all possible products
- Must cope with **overflow**
 - Because frequently want 32-bit product as result of multiplying two 32-bit numbers

Binary Multiplication

- With only two choices, each step of multiplication is simple:
 1. Place a copy of the multiplicand (1 multiplicand) in the proper place if the multiplier bit is a 1
 2. Place 0 (0 multiplicand) in the proper place if the bit is 0

Example

Multiplicand		1	1	0	1	1
Multiplier	x		1	0	1	0

Binary Multiplication

Multiplicand			1	1	0	1	1
Multiplier	x			1	0	1	0
			0	0	0	0	0
Product							

Binary Multiplication

Multiplicand			1	1	0	1	1
Multiplier	x			1	0	1	0
			0	0	0	0	0
		1	1	0	1	1	
Product							

Binary Multiplication

Multiplicand		1	1	0	1	1
Multiplier	×		1	0	1	0
		0	0	0	0	0
		1	1	0	1	1
	0	0	0	0	0	
Product						

Binary Multiplication

Multiplicand				1	1	0	1	1
Multiplier	x				1	0	1	0
				0	0	0	0	0
			1	1	0	1	1	
		0	0	0	0	0		
		1	1	0	1	1		
Product								

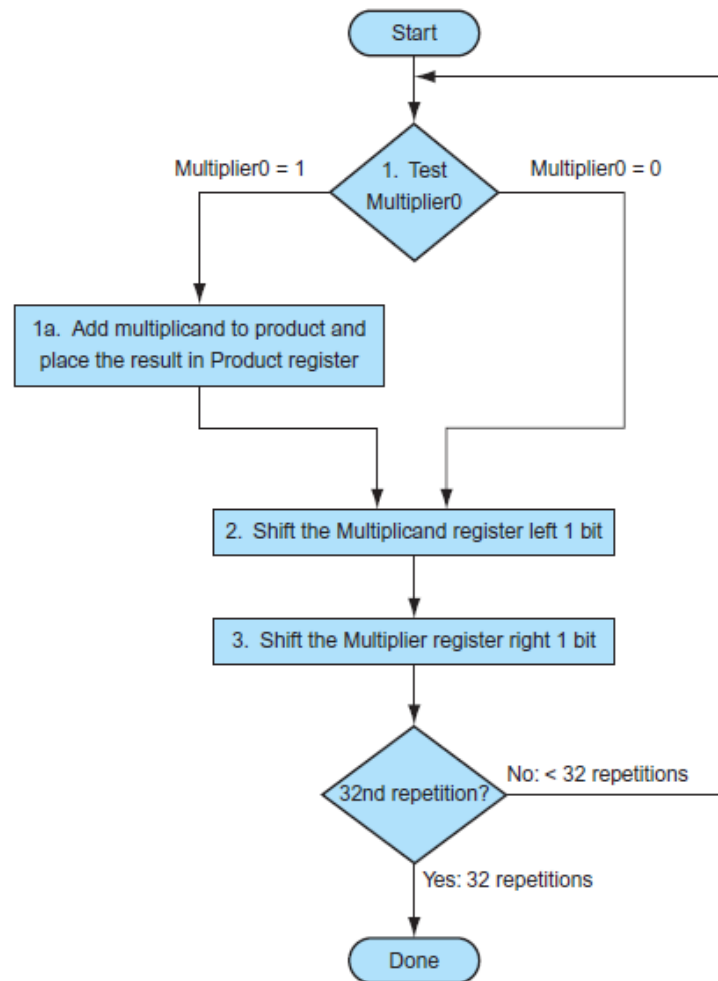
Binary Multiplication

Multiplicand				1	1	0	1	1	
Multiplier	x				1	0	1	0	
					0	0	0	0	0
				1	1	0	1	1	
			0	0	0	0	0		
		1	1	0	1	1			
Product	1	0	0	0	0	1	1	1	0

Algorithm

- From example, will need to move multiplicand left one digit each step, as it may be added to intermediate products
 - 32-bit multiplicand would move 32 bits to left
- Hence, we need a 64-bit **Multiplicand register**, initialized with the 32-bit multiplicand in right half and zero in left half
- Register is then shifted left 1 bit each step to align multiplicand with sum being accumulated in the 64-bit **Product register**

Algorithm



Algorithm

- Three basic steps needed for each bit
 1. LSB of multiplier (Multiplier0) determines whether multiplicand is added to **Product** register
 2. Left shift has effect of moving intermediate operands to left
 3. Shift right next bit of multiplier to examine in the following iteration
- Steps are repeated 32 times to obtain the product

Performance

- If each step took a clock cycle, algorithm would require almost 100 clock cycles to multiply two 32-bit numbers
- Relative importance of arithmetic operations like multiply varies with program, but addition and subtraction may be anywhere from 5 to 100 times more popular than multiply
- Accordingly, in many applications, multiply can take clock cycles without significantly affecting performance

Speed-up

- Algorithm is easily refined to take 1 clock cycle per step
- Speed-up comes from performing operations in parallel:
 - Multiplier and multiplicand are shifted while multiplicand is added to product if multiplier bit is a 1
 - HW just has to ensure that it tests right bit of multiplier and gets the preshifted version of the multiplicand
 - HW is usually further optimized to halve width of adder and registers by noticing where there are unused portions of registers and adders

Example

- Using 4-bit numbers, multiply $2_{\text{ten}} \times 3_{\text{ten}}$, or $0010_{\text{two}} \times 0011_{\text{two}}$.
- Figure shows value of each register for each of steps labeled according to flowchart, with final value of $0000\ 0110_{\text{two}}$ or 6_{ten}
- Color is used to indicate register values that change
- Bit circled is the one examined to determine operation of next step

Example (cont.)

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	0011	0000 0010	0000 0000
1				
2				
3				
4				

Example (cont.)

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	0011	0000 0010	0000 0000
1	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0011	0000 0010	0000 0010
2				
3				
4				

Example (cont.)

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	0011	0000 0010	0000 0000
1	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0011	0000 0010	0000 0010
	2: Shift left Multiplicand	0011	0000 0100	0000 0010
2				
3				
4				

Example (cont.)

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	001 ^①	0000 0010	0000 0000
1	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0011	0000 0010	0000 0010
	2: Shift left Multiplicand	0011	0000 0100	0000 0010
	3: Shift right Multiplier	000 ^①	0000 0100	0000 0010
2				
3				
4				

Example (cont.)

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	0011	0000 0010	0000 0000
1	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0011	0000 0010	0000 0010
	2: Shift left Multiplicand	0011	0000 0100	0000 0010
	3: Shift right Multiplier	0001	0000 0100	0000 0010
2	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0001	0000 0100	0000 0110
	2: Shift left Multiplicand	0001	0000 1000	0000 0110
	3: Shift right Multiplier	0000	0000 1000	0000 0110
3				
4				

Example (cont.)

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	0011①	0000 0010	0000 0000
1	1a: 1 $\Rightarrow$ Prod = Prod + Mcand	0011	0000 0010	0000 0010
	2: Shift left Multiplicand	0011	0000 0100	0000 0010
	3: Shift right Multiplier	000①	0000 0100	0000 0010
2	1a: 1 $\Rightarrow$ Prod = Prod + Mcand	0001	0000 0100	0000 0110
	2: Shift left Multiplicand	0001	0000 1000	0000 0110
	3: Shift right Multiplier	000①	0000 1000	0000 0110
3	1: 0 $\Rightarrow$ No operation	0000	0000 1000	0000 0110
	2: Shift left Multiplicand	0000	0001 0000	0000 0110
	3: Shift right Multiplier	000①	0001 0000	0000 0110
4				

Example (cont.)

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	001 <u>1</u>	0000 0010	0000 0000
1	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0011	0000 0010	0000 0010
	2: Shift left Multiplicand	0011	0000 0100	0000 0010
	3: Shift right Multiplier	000 <u>1</u>	0000 0100	0000 0010
2	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0001	0000 0100	0000 0110
	2: Shift left Multiplicand	0001	0000 1000	0000 0110
	3: Shift right Multiplier	000 <u>0</u>	0000 1000	0000 0110
3	1: $0 \Rightarrow$ No operation	0000	0000 1000	0000 0110
	2: Shift left Multiplicand	0000	0001 0000	0000 0110
	3: Shift right Multiplier	000 <u>0</u>	0001 0000	0000 0110
4	1: $0 \Rightarrow$ No operation	0000	0001 0000	0000 0110
	2: Shift left Multiplicand	0000	0010 0000	0000 0110
	3: Shift right Multiplier	0000	0010 0000	0000 0110

Signed Multiplication

- Convert multiplier and multiplicand to positive and remember signs
- Algorithm should run for 31 iterations, leaving signs out of calculation
- Negate product only if original signs disagree
- Shifting steps would need to extend sign of product for signed numbers
- When algorithm completes, lower word would have 32-bit product

Multiply in MIPS

- MIPS provides a separate pair of 32-bit registers to contain the 64-bit product
- Registers are called
 - *Hi* and
 - *Lo*

64-bit product



Multiply Instructions

- To produce a properly signed or unsigned product, MIPS has two instructions
 - multiply (`mult`)
 - multiply unsigned (`multu`)

`mult rs, rt` # signed: Product in [Hi,Lo]

`multu rs, rt` # unsigned: Product in [Hi,Lo]

Multiply Instructions

- To fetch integer 32-bit **lo-product**, use
 - *move from lo* (**mfl****o**)
- To fetch integer 32-bit **hi-product**, use
 - *move from lo* (**mfh****i**)

Multiply Instructions

- MIPS assembler generates a pseudoinstruction for multiply that specifies three general-purpose registers to place product into registers
- Result is only 32-bits

`mul rd, rs, st` #low-order 32 bits of product